

Full Stack Development with MERN

Project Documentation

Date:	03 October 2026
Team ID:	SWTID-2026-3636
Project Title:	Implement Client Script & UI Policy (Incident Table)
Stack:	MERN Stack (MongoDB, Express.js, React.js, Node.js)

1. Introduction

- **Project Title:** Implement Client Script & UI Policy (Incident Table)
- **Team Members & Roles:**
 1. Deepesh Harshan HD - Team Lead
 2. Jayapriyan N
 3. Hemesh S
 4. Kanishkar P

2. Project Overview

- **Purpose:** The primary purpose of this project is to build an IT Service Management (ITSM) Incident workspace that dynamically controls form field behavior—making fields mandatory, read-only, or visible based on specific state and priority conditions. This reduces form clutter, speeds up incident triage, and eliminates missing resolution details upon ticket closure.
- **Features & Functionalities:**
 1. Dynamic UI Field Visibility: Automatically reveals 'Resolution Code' and 'Resolution Notes' only when ticket state is set to 'Resolved'.
 2. Dynamic Mandatory Field Rules: Forces 'Work Notes' and 'Assignment Group' when Incident Priority is Critical (P1) or High (P2).
 3. Form Security & State Lock: Automatically locks all core fields to Read-Only once an incident reaches 'Closed' state.
 4. Real-time Priority Auto-Calculation: Calculates Priority dynamically on the client side based on Impact and Urgency selections.
 5. Interactive Guidance: Displays diagnostic troubleshooting prompts when selecting specific categories like Hardware or Network.

3. Architecture

- **Frontend Architecture (React.js):** Built with React.js using component-driven UI architecture, Context API / Redux for state management, Axios for asynchronous API requests, and custom hooks for client-side field validation and dynamic UI policy evaluation.
- **Backend Architecture (Node.js & Express.js):** RESTful API backend designed with Node.js and Express.js, implementing MVC (Model-View-Controller) structure, middleware for JWT authentication, request validation, and centralized error handling.
- **Database Architecture (MongoDB):** NoSQL database using Mongoose ODM to model flexible schemas for Incident records, User roles, and dynamic UI Policy rules with optimized indexing on ticket state and priority fields.

4. Setup Instructions

- **Prerequisites:** Node.js (v18.x or higher), npm (v9.x or higher), MongoDB (v6.x or MongoDB Atlas Cloud instance), Git.
- **Step-by-Step Installation:**
 1. Clone Repository: `git clone https://github.com/Deepeshharshan/Implement-Client`
 2. Install Client Dependencies: `cd client && npm install`
 3. Install Server Dependencies: `cd ../server && npm install`
 4. Set Environment Variables: Create a `.env` file in the server directory with:


```
PORT=5000
MONGO_URI=mongodb://localhost:27017/incident_db
JWT_SECRET=your_jwt_secret_key
```

5. Folder Structure

```
project-root/
├── client/           # React Frontend
│   ├── public/      # Static assets & index.html
│   └── src/
│       ├── components/ # IncidentForm, UIPolicyRules, IncidentList
│       ├── hooks/      # useUIPolicy.js, useClientScript.js
│       ├── context/    # AuthContext.js, IncidentContext.js
│       └── App.js      # Main React App Component
└── server/          # Node.js & Express Backend
    ├── config/        # db.js (MongoDB Connection)
    ├── controllers/    # incidentController.js, uiPolicyController.js
    ├── middleware/     # authMiddleware.js, errorHandler.js
    ├── models/         # Incident.js, UIPolicy.js, User.js
    ├── routes/         # incidentRoutes.js, uiPolicyRoutes.js
    └── server.js       # Express App Entry Point
```

6. Running the Application

- Start Backend Server: Open terminal in /server directory and run:
npm start (or npm run dev with nodemon)
- Start Frontend Application: Open terminal in /client directory and run:
npm start
- Access Application: Navigate to http://localhost:3000 in your web browser.

7. API Documentation

Method	Endpoint	Description	Auth Required
GET	/api/incidents	Fetch all incident records with pagination	Yes (JWT)
GET	/api/incidents/:id	Fetch single incident record details	Yes (JWT)
POST	/api/incidents	Create a new incident record	Yes (JWT)
PUT	/api/incidents/:id	Update incident (validates mandatory resolution fields)	Yes (JWT)
GET	/api/ui-policies	Fetch active UI policies and field rules	Yes (JWT)
POST	/api/auth/login	User login & JWT token generation	No

8. Authentication & Authorization

Authentication is implemented using JSON Web Tokens (JWT). Users authenticate via /api/auth/login and receive a signed JWT token, which is stored securely on the client side and passed in the Authorization header (Bearer <token>) for subsequent REST requests.

Role-Based Access Control (RBAC) middleware enforces permissions, ensuring that only users with 'itil' or 'incident_manager' roles can update incident fields or alter UI policy rules.

9. User Interface & Feature Walkthrough

- Incident Workspace Form: Modern React dashboard featuring real-time form validation, state dropdown listeners, and dynamic field toggling.
- Dynamic Visibility in Action: When State is updated to 'Resolved', 'Resolution Code' and 'Resolution Notes' smoothly fade into view.
- Closed Ticket Lock: Selecting 'Closed' state locks all input fields to Read-Only with visual lock icons.
- [Insert Screenshots / GIFs of Incident Workspace Here]

10. Testing Strategy & Execution

- Unit Testing: Jest and React Testing Library used to test individual form components and custom UI policy hooks.
- Integration Testing: Supertest used to test Express API endpoints and MongoDB database transactions.

- User Acceptance Testing (UAT): Service Desk Analysts performed UAT validation across 50+ test cases covering state transitions and priority calculations.

11. Screenshots or Demo Link

- Live Application Demo Link: <https://incident-ui-policy-mern.demo.app>
- Video Walkthrough Link: <https://youtube.com/demo-walkthrough-link>
- GitHub Repository: <https://github.com/Deepeshharshan/Implement-Client>

12. Known Issues

1. Legacy Browser Animation Lag: Minor UI animation delay during rapid state dropdown toggling on older Internet Explorer / legacy browsers.
2. Multi-Tab Session Sync: State updates in one browser tab do not instantly reflect in open duplicate tabs without manual refresh.

13. Future Enhancements

1. Machine Learning Incident Categorization: Integrate AI-driven auto-categorization to recommend category and urgency upon entering short descriptions.
2. Real-Time WebSocket Collaboration: Enable Socket.io to allow multiple support agents to view live updates on shared incident records simultaneously.
3. Mobile PWA Support: Develop Progressive Web App support for offline incident drafting and mobile push notifications.